WPS AI Agent CLI Paragraph Scoped

产品需求文档（PRD）

文档版本	v1.1
日期	2026-10-02
状态	更新版（融入后续改进与进化方向）
作者	项目架构组
基于版本	v1.0 + 后续改进与进化方向讨论

1. 文档信息与变更记录

版本	日期	变更说明	作者
v1.0	2026-10-02	初始版本，整合项目目标、功能规划、技术路线与验证要求	架构组
v1.1	2026-10-02	新增「后续改进与进化方向」章节；细化短中长期路线；强化验证闭环、会话服务、HTML 三模式与可观测性要求；更新交付计划与风险	架构组

2. 产品概述

2.1 产品愿景

开发一个面向 AI Agent 的 Windows 桌面版 WPS CLI（主入口）+ 可选 MCP 服务，使具备本地工具执行能力的 Agent（Claude Code、Codex、Cursor、本地工具链等）能够通过结构化、确定性命令，安全、可靠地操作 WPS 文字、表格、演示，完成现有文档与模板的读取、修改、计算、格式转换、结果验证与文件交付。

CLI 是主要入口；MCP 复用同一核心逻辑。明确区分 ChatGPT 网页端的工具连接机制，不假设网页端可直接执行本机命令。

2.2 核心价值

- 真实引擎操作：使用 WPS 真实计算、排版与导出能力，而非纯文件解析。
- 可验证与可恢复：命令成功 ≠ 内容/排版正确；强制备份、状态查询、验证钩子。
- Agent 友好：结构化 JSON 输入输出、稳定 document_id、能力按需查询、幂等与超时处理。
- 明确边界：区分「控制正在运行的 WPS」「使用 WPS 打开/计算/导出」「离线文件库读写」「浏览器渲染 HTML」。

2.3 目标用户与场景

主要用户：本地 AI Agent 开发者与使用者（需要操作 WPS 文档的自动化流水线）。

核心场景：

- 模板填充与批量报告生成
- 现有文档定位修改与格式保留编辑
- 表格公式写入 + 真实计算后核对
- 演示模板填充与对象级替换
- 格式转换与批量交付
- Agent 规划 → 执行 → 验证 → 修正闭环

2.4 产品边界（必须明确）

- 首版仅支持 Windows 桌面版 WPS 文字、表格、演示。
- 文件可被 WPS 打开 ≠ 控制了 WPS。
- 写入公式 ≠ 已计算。
- 命令成功 ≠ 内容或排版正确。
- 不默认开放任意 Python/宏/脚本执行。
- PDF 反向转换（OCR + 结构重建）单独评估，不视为普通另存为。

3. 目标与成功指标

3.1 产品目标

1. 提供确定性、可脚本化的 CLI，支持 Agent 可靠调用。
2. 覆盖三组件核心读写、计算、格式保留与验证。
3. 建立实际支持的转换矩阵，明确后端、前提、保留能力与损失。
4. 实现可恢复执行与结果可验证。
5. 通过 Phase 0 可行性验证后再进入大规模实现。

3.2 成功指标（验收导向）

- 目标 WPS 版本矩阵下核心操作成功率达到预设阈值。
- 公式计算后原始值/显示值一致性验证通过。
- 格式保留指标（结构、样式抽查）达标。
- 所有修改操作均有可恢复备份。
- Agent 端到端流程可完整跑通（检查 → 规划 → 执行 → 验证 → 交付）。
- 错误码覆盖完整，执行后端明确标注。

4. 功能需求

4.1 功能矩阵

类别	首版（MVP）	后续	暂缓及理由
公共能力	版本/ProgID/能力检测与 fallback；连接/打开/列出/保存副本/关闭；稳定 document_id；批量与任务状态；超时；修改前强制备份；基础结果验证	更强会话持久与并发控制；干跑模式	任意脚本/宏默认开放
文字	读取正文/段落/标题结构/表格/书签/基础样式；定位替换、插入、书签填充、表格读写、基础格式修改；尽量保留现有结构	高级样式、目录、页眉页脚	全文重建
表格	工作表管理；区域批量读写；公式写入+真实计算后核对；区分原始值/公式/显示文本/计算结果；多字母列、日期、空值、中文	数据透视、图表、条件格式深度	完整 Power Query/VBA 对等
演示	读取页面/形状/占位符/文本/图片；精确定位替换文本/图片、模板填充；保留未修改对象与主题	高级动画/主题	复杂交互完整保留
格式转换	核心方向矩阵+批量报告；明确后端与损失；HTML 视觉优先基线	跨组件互转；HTML 可编辑优先与受控往返	PDF 反向（OCR 单独评估）
Agent 支持	结构化 JSON I/O、参数校验、错误码、能力按需查询、请求 ID/幂等、验证结果输出	长任务进度流、自适应恢复、干跑预览	默认开放任意代码执行

4.2 公共能力详细要求

- 检测 WPS 版本、接口注册、组件及操作能力，启动时主动提示支持情况。
- 使用稳定 document_id 定位目标文档，避免依赖活动窗口。
- 修改前强制备份；支持任务状态查询；超时后先查状态再决策。
- 输出必须包含：执行后端、修改摘要、验证结果。
- 支持干跑（dry-run）模式，供 Agent 预览操作影响。

4.3 格式转换要求

建立实际支持矩阵，不承诺任意无损互转。每个方向明确：后端（WPS COM / 文件库 / 浏览器）、前提条件、保留能力与已知损失。

HTML 高保真三种模式：

1. 视觉优先：浏览器渲染到 PDF/图片，以外观一致为主（首版优先落地）。
2. 可编辑优先：HTML/CSS 映射到原生段落、表格、形状、图片。
3. 受控往返：自有 HTML 规范 + 对象 ID + 映射元数据。

必须处理字体、图片、CSS、相对路径、资源缺失、页面尺寸、分页、不支持 CSS/脚本/动画，并定义降级行为与测试指标（视觉一致性、内容完整性、可编辑性、结构保留）。

4.4 Agent 调用设计

推荐流程：检查环境与文档 → 制定步骤 → 执行 → 保存与渲染 → 验证 → 修正或报告。

接口要求：

- 结构化 JSON 输入输出 + 参数校验 + 明确错误码。
- 按需查询命令与能力（控制上下文成本）。
- 批量读写、分页读取、长任务状态查询。

- 请求标识与重复执行防护；支持干跑预览。
- 输出文件、执行后端、修改摘要、验证结果。

5. 非功能需求
- 可靠性：命令成功必须配合验证；强制备份；可恢复。
- 确定性：相同输入在相同环境下产生可预期结果。
- 可观测性：完整错误码、执行后端标注、修改摘要、验证结果；结构化日志与执行轨迹。
- 性能：批量操作高效；超时可控；状态查询优先于盲目重试。
- 安全性：首版不默认开放任意代码执行；修改操作可审计。
- 兼容性：明确支持的 WPS 版本矩阵；ProgID fallback；不假设 MS Office 兼容性直接适用。
- 可维护性：分层架构，CLI 与 MCP 共用核心逻辑。

6. 技术架构与选型

6.1 推荐架构

```
CLI / MCP 入口 → 会话与任务服务（document_id、状态、超时、幂等、备份） → 共用操作核心（参数校验、错误码、验证钩子） → WPS 后端适配器（COM 优先 + ProgID fallback） + 格式转换核心（ExportAsFixedFormat/SaveAs2 + HTML 解析/浏览器渲染 + 质量验证）
```

6.2 技术选型原则
- COM 与 JSAPI 均需验证；GUI 自动化仅作局部补充。
- 优先参考：yb2460/harness-anything（CLI + COM）、sbroenne/mcp-server-excel（会话与 CLI/MCP 共用核心）、lewis-hui1202/WPS-AI（JSAPI + MCP）。
- 所有兼容性、精度、成功率声明必须经过实测验证。

6.3 待验证假设（Phase 0 必须完成）
6. 目标 WPS 版本 ProgID 注册与 COM 可用性（含近期失效报告）。
7. 公式计算与显示值一致性。
8. ExportAsFixedFormat / SaveAs2 参数与实际损失。
9. 多文档/多组件并发稳定性。
10. HTML 映射保真度与降级边界。

7. 用户故事与端到端流程

典型 Agent 流程：

11. 检查环境与文档状态。
12. 制定可执行步骤（可含干跑预览）。
13. 带备份与请求 ID 执行。
14. 保存副本并渲染。
15. 执行验证（内容、计算、格式）。
16. 修正或输出问题报告。

示例用户故事：
- 作为 Agent，我希望通过稳定 document_id 定位并修改现有文档的指定段落，并获得格式保留验证结果。
- 作为 Agent，我希望写入表格公式后触发真实计算，并核对原始值与显示值是否一致。
- 作为 Agent，我希望批量转换目录下文档，并获得逐文件状态、错误与输出报告。
- 作为 Agent，我希望在执行破坏性操作前先进行干跑预览，确认影响范围。

8. 交付计划与里程碑

阶段	内容	产出
Phase 0	技术调研与可行性验证（ProgID 矩阵、最小原型、转换损失、HTML 基线）	可行性报告 + 版本矩阵
Phase 1	MVP CLI（公共+三组件核心+备份/验证+基础转换+JSON 协议+干跑）	可运行 CLI + 测试样例
Phase 2	会话服务、MCP 复用、批量、HTML 三模式、质量验证、可观测性	完整 Agent 可用版本
Phase 3	跨组件、高级恢复、性能优化、测试体系与生态扩展	生产就绪版本

依赖：Windows + 目标 WPS 版本、pywin32（或等效）、可选 Node（MCP）、浏览器引擎（HTML 视觉模式）。

9. 后续改进与进化方向

9.1 短期（Phase 1 完成后，1-2 个月）

目标：把 MVP 打磨成「可被 Agent 稳定调用」的版本。

1. 验证闭环强化：完善「修改前备份 + 执行后验证」，增加内容哈希、结构快照、计算结果对比；建立标准化错误码与恢复建议。
2. 版本兼容矩阵落地：完成主流 WPS 版本 ProgID 与 COM 可用性实测，形成官方支持矩阵 + 不支持列表。
3. Agent 体验优化：能力查询支持按场景过滤；增加干跑模式；请求 ID + 幂等保护完善。
4. 转换质量基线：为核心转换方向建立最小测试样例集与指标；HTML 视觉优先模式优先落地。

9.2 中期（Phase 2，3-6 个月）

目标：从「能用」升级到「好用且可规模化」。

5. 会话与任务服务成熟：常驻会话管理，支持多文档并行与长时间任务状态查询；完善 document_id 生命周期与资源回收。
6. MCP 深度集成：CLI 与 MCP 完全共用核心逻辑；提供标准工具描述与示例配置。
7. HTML 高保真三模式完善：可编辑优先与受控往返落地；建立自有 HTML 规范 + 对象 ID 映射。
8. 批量与效率：目录/文件清单批量操作 + 逐文件状态报告；长任务进度流与可中断设计。
9. 可观测性与运维：结构化日志 + 执行轨迹；本地诊断命令。

9.3 长期（Phase 3 及以后）

目标：成为 Windows 上 WPS 自动化的可靠基础设施。

10. 跨组件与高级能力：文字表格↔工作表、大纲→演示等跨组件转换；有选择支持数据透视、图表等（充分验证后）。
11. 更强的可恢复与安全模型：明确备份恢复、应用撤销、事务回滚边界；操作审计日志；细粒度权限控制。
12. 质量与测试体系：自动化回归测试套件；「精确转换」指标持续迭代；与真实 Agent 的端到端评测流水线。
13. 生态扩展（谨慎）：评估 JSAPI 加载项作为补充入口；考虑与其他办公软件的统一抽象层（绝不假设完全兼容）。

9.4 技术进化重点

方向	关键动作	优先级
COM 稳定性	ProgID fallback + 版本检测 + 进程生命周期管理	最高
验证体系	内容/计算/格式三层验证 + 可机器解析结果	最高
会话模型	常驻服务 + document_id + 状态查询	高
HTML 保真	三模式 + 指标驱动 + 显式降级	高
可观测性	结构化日志 + 执行轨迹 + 诊断命令	中高
测试资产	版本矩阵 + 用户故事回归 + Agent 端到端评测	中高
安全边界	默认最小权限 + 审计 + 明确不支持列表	持续

9.5 进化原则（必须遵守）

- 先验证，再实现：任何新功能必须有对应的可行性验证。
- 明确边界：持续强化「命令成功 ≠ 内容正确」「打开 ≠ 控制」「写入公式 ≠ 已计算」。
- Agent 优先：所有接口设计以「可被 LLM 可靠调用」为第一目标。
- 可回退：任何破坏性操作必须有备份与恢复路径。
- 信息诚实：兼容性、精度、成功率必须基于实测，不编造。

10. 风险与缓解

风险	影响	缓解措施
WPS 版本 COM 接口差异/失效	核心功能不可用	强制版本矩阵 + ProgID fallback + 明确不支持列表
计算与显示值不一致	验证失败	强制计算后核对钩子
HTML 保真度不足	用户预期落空	多模式 + 指标 + 显式降级
进程泄漏/并发问题	稳定性差	生命周期管理 + document_id + 状态查询
Agent 误用导致不可恢复修改	数据风险	强制备份 + 错误码 + 能力查询 + 干跑模式

11. 相比现有项目的具体改进
    - 相对 harness-anything：增加稳定 document_id、强制备份、计算结果与格式验证、明确执行后端与错误码、Agent 可恢复流程、HTML 多模式与质量指标、干跑模式；主动应对近期 COM 版本问题。
    - 相对 WPS-AI：提供独立无 UI CLI 主入口、更强会话与幂等、转换损失矩阵、离线降级路径。
    - 相对 mcp-server-excel：扩展至 WPS 三组件、统一验证与 document_id、HTML 专项设计。
    - 相对纯文件库方案：真实引擎计算与排版，明确「打开 ≠ 控制」「写入公式 ≠ 已计算」「命令成功 ≠ 内容正确」。
12. 附录
    - 参考项目：yb2460/harness-anything、lewis-hui1202/WPS-AI、sbroenne/mcp-server-excel。
    - 关键原则：功能数量服从可靠性；信息不足时明确假设与验证方法；不编造兼容性、精度或成功率。
    - 下一步行动：立即启动 Phase 0 可行性验证，产出版本矩阵与最小原型报告；根据验证结果更新本 PRD 至 v1.2。

— 文档结束 —

此 PRD v1.1 已融入后续改进与进化方向，可作为详细设计与任务拆解的基础。